

MASTER DATASET (Use for All Problems)

Customers

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  city VARCHAR(50),  
  signup_date DATE  
);
```

```
INSERT INTO customers VALUES  
(1,'Raj','Mumbai','2024-01-10'),  
(2,'Priya','Pune','2024-02-15'),  
(3,'Amit','Delhi','2024-03-01'),  
(4,'Sneha','Chennai','2024-03-20'),  
(5,'John','Bangalore','2024-04-01');
```

Products

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY,  
  product_name VARCHAR(100),  
  category VARCHAR(50),  
  price DECIMAL(10,2)  
);
```

```
INSERT INTO products VALUES  
(101,'Laptop','Electronics',60000),  
(102,'Mobile','Electronics',30000),  
(103,'Chair','Furniture',5000),  
(104,'Table','Furniture',8000),  
(105,'Monitor','Electronics',15000);
```

Orders

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,
```

```
order_date DATE,  
total_amount DECIMAL(10,2)  
);
```

```
INSERT INTO orders VALUES  
(1001,1,'2025-01-01',60000),  
(1002,2,'2025-01-03',30000),  
(1003,1,'2025-01-05',15000),  
(1004,3,'2025-01-10',5000),  
(1005,4,'2025-01-15',8000),  
(1006,2,'2025-01-20',45000);
```

Order Items

```
CREATE TABLE order_items (  
order_item_id INT PRIMARY KEY,  
order_id INT,  
product_id INT,  
quantity INT  
);
```

```
INSERT INTO order_items VALUES  
(1,1001,101,1),  
(2,1002,102,1),  
(3,1003,105,1),  
(4,1004,103,1),  
(5,1005,104,1),  
(6,1006,102,1),  
(7,1006,105,1);
```

SECTION 1 : SELECT + WHERE

Problem 1

Find customers from Mumbai.

Problem 2

Find products costing more than ₹20,000.

Problem 3

Find orders placed after Jan 10, 2025.

SECTION 2 : SELECT + ORDER BY

Problem 4

Show products from highest to lowest price.

Problem 5

Show customers sorted by signup date.

Problem 6

Display orders sorted by amount descending.

SECTION 3 : SELECT + AGGREGATE FUNCTIONS

Problem 7

Find total sales.

Problem 8

Find average order value.

Problem 9

Find maximum order amount.

Problem 10

Find minimum product price.

SECTION 4 : GROUP BY

Problem 11

Find total revenue generated by each customer.

Problem 12

Find total quantity sold per product.

Problem 13

Count customers city-wise.

SECTION 5 : GROUP BY + HAVING

Problem 14

Show customers whose purchases exceed ₹50,000.

Problem 15

Show cities having more than 1 customer.

Problem 16

Show products sold more than once.

SECTION 6 : INNER JOIN

Problem 17

Show customer names with their orders.

Problem 18

Show product names for each order.

Problem 19

Show customer name and order amount.

SECTION 7 : JOIN + GROUP BY

Problem 20

Find total spending by customer.

Problem 21

Find total sales by product category.

Problem 22

Find order count per customer.

SECTION 8 : JOIN + GROUP BY + HAVING

Problem 23

Customers spending more than ₹40,000.

Problem 24

Categories generating more than ₹50,000 revenue.

Problem 25

Customers placing more than one order.

SECTION 9 : SUBQUERY

Problem 26

Find customers whose spending exceeds average spending.

Problem 27

Find products priced above average product price.

Problem 28

Find orders above average order amount.

SECTION 10 : CORRELATED SUBQUERY

Problem 29

Find highest order of each customer.

Problem 30

Find most expensive product in each category.

Problem 31

Find latest order per customer.

SECTION 11 : DERIVED TABLE

Problem 32

Find customers whose total spending exceeds overall average customer spending.

Hint:

SELECT *

```
FROM  
(  
    -- aggregate query  
) x  
WHERE ...
```

Problem 33

Find categories with revenue above average category revenue.

SECTION 12 : CTE

Problem 34

Find top-spending customer.

Problem 35

Find category-wise revenue.

Problem 36

Find top 3 highest revenue categories.

SECTION 13 : CASE EXPRESSION

Problem 37

Classify products:

>50000 Premium
20000-50000 Standard
Below 20000 Budget

Problem 38

Classify customers:

>50000 Gold

20000-50000 Silver

Else Bronze

SECTION 14 : STRING FUNCTIONS

Problem 39

Convert customer names to uppercase.

Problem 40

Show first 3 characters of product names.

Problem 41

Concatenate customer name and city.

SECTION 15 : DATE FUNCTIONS

Problem 42

Find number of days since customer signup.

Problem 43

Find month-wise sales.

Problem 44

Find customers joined in March.

SECTION 16 : COALESCE

Create some NULL cities and solve:

Problem 45

Replace NULL city with 'Unknown'.

Problem 46

Display customer city using COALESCE.

SECTION 17 : WINDOW FUNCTIONS

ROW_NUMBER

Problem 47

Assign serial number to orders by amount.

Problem 48

Assign serial number within each customer.

RANK

Problem 49

Rank customers by total spending.

Problem 50

Rank products by price.

DENSE_RANK

Problem 51

Rank products without gaps.

NTILE

Problem 52

Divide customers into 4 spending groups.

SECTION 18 : PARTITION BY

Problem 53

Rank orders within each customer.

Problem 54

Find highest order per customer.

Problem 55

Find running total of orders per customer.

SECTION 19 : LAG

Problem 56

Show previous order amount for every order.

Problem 57

Show previous order date.

Problem 58

Calculate difference from previous order.

SECTION 20 : LEAD

Problem 59

Show next order amount.

Problem 60

Show next order date.

Problem 61

Calculate gap to next order.

SECTION 21 : FIRST_VALUE

Problem 62

Show customer's first order amount.

Problem 63

Show customer's first order date.

SECTION 22 : LAST_VALUE

Problem 64

Show latest order amount.

Problem 65

Show latest order date.

SECTION 23 : RUNNING TOTAL

Problem 66

Calculate cumulative revenue.

Problem 67

Calculate customer-wise cumulative spending.

SECTION 24 : PERCENT_RANK

Problem 68

Find percentile ranking of customer spending.

SECTION 25 : CUME_DIST

Problem 69

Find cumulative distribution of product prices.

SECTION 26 : ROLLUP

Problem 70

Generate:

Category Revenue

Product Revenue

Grand Total

using ROLLUP.

SECTION 27 : CUBE

Problem 71

Generate revenue by:

Category

Customer

Category+Customer

Grand Total

SECTION 28 : GROUPING SETS

Problem 72

Generate reports:

Category

Customer

Month

using a single query.

SECTION 29 : ADVANCED INTERVIEW QUESTIONS

Problem 73

Top 3 customers by spending.

Problem 74

Second highest order amount.

Problem 75

Third most expensive product.

Problem 76

Customer contributing highest revenue.

Problem 77

Most sold product.

Problem 78

Customer with longest inactivity.

Problem 79

Month-over-month revenue growth.

Problem 80

Revenue contribution percentage of each customer.

SECTION 30 : REAL-TIME BUSINESS PROBLEMS

Problem 81 (Amazon)

Top-selling products in Electronics category.

Problem 82 (Flipkart)

Customers who haven't ordered in last 30 days.

Problem 83 (Banking)

Find top 5 customers by transaction volume.

Problem 84 (Hospital)

Find doctor treating most patients.

Problem 85 (Manufacturing)

Find product having highest production cost.

Problem 86 (Telecom)

Find customer generating highest monthly revenue.

Problem 87 (Insurance)

Find policies expiring this month.

Problem 88 (HR)

Find department with highest salary expense.

Problem 89 (Retail)

Find repeat customers.

Problem 90

Find customers purchasing products from multiple categories.

Problem 91

Find products never ordered.

Problem 92

Find customers with no orders.

Problem 93

Find products ordered by every customer.

Problem 94

Find most profitable category.

Problem 95

Find monthly cumulative revenue.

Problem 96

Find moving average of revenue.

Problem 97

Find revenue trend month-over-month.

Problem 98

Find customer retention percentage.

Problem 99

Find top product per category.

Problem 100

Build a complete sales dashboard query showing:

- Total Revenue
 - Total Customers
 - Total Orders
 - Average Order Value
 - Top Customer
 - Top Product
 - Best Category
-

This 100-problem workbook covers virtually every SELECT-based SQL combination used in interviews and real-world projects:

SELECT
WHERE
ORDER BY
GROUP BY
HAVING
JOINS
SUBQUERIES
CORRELATED SUBQUERIES
DERIVED TABLES
CTEs
CASE
STRING FUNCTIONS
DATE FUNCTIONS
COALESCE
WINDOW FUNCTIONS
ROW_NUMBER
RANK
DENSE_RANK
NTILE
LAG
LEAD
FIRST_VALUE
LAST_VALUE
PERCENT_RANK
CUME_DIST
ROLLUP
CUBE
GROUPING SETS
RUNNING TOTALS
TOP-N ANALYSIS
BUSINESS KPI REPORTING

PANEL 2: SOLUTIONS (PROBLEMS 1–25)

Problem 1

Find customers from Mumbai

```
SELECT *  
FROM customers  
WHERE city = 'Mumbai';
```

Problem 2

Products costing more than ₹20,000

```
SELECT *  
FROM products  
WHERE price > 20000;
```

Problem 3

Orders placed after Jan 10, 2025

```
SELECT *  
FROM orders  
WHERE order_date > '2025-01-10';
```

Problem 4

Products highest to lowest price

```
SELECT *  
FROM products  
ORDER BY price DESC;
```

Problem 5

Customers by signup date

```
SELECT *  
FROM customers  
ORDER BY signup_date;
```

Problem 6

Orders by amount descending

```
SELECT *  
FROM orders  
ORDER BY total_amount DESC;
```

Problem 7

Total Sales

```
SELECT SUM(total_amount) AS total_sales  
FROM orders;
```

Problem 8

Average Order Value

```
SELECT AVG(total_amount) AS avg_order_value  
FROM orders;
```

Problem 9

Maximum Order Amount

```
SELECT MAX(total_amount) AS highest_order  
FROM orders;
```

Problem 10

Minimum Product Price

```
SELECT MIN(price) AS lowest_price  
FROM products;
```

Problem 11

Revenue Generated By Each Customer

```
SELECT  
customer_id,  
SUM(total_amount) AS revenue  
FROM orders  
GROUP BY customer_id;
```

Problem 12

Quantity Sold Per Product

```
SELECT  
product_id,  
SUM(quantity) total_qty  
FROM order_items  
GROUP BY product_id;
```

Problem 13

Customer Count By City

```
SELECT  
city,  
COUNT(*) customer_count  
FROM customers  
GROUP BY city;
```

Problem 14

Customers Spending More Than ₹50,000

```
SELECT  
customer_id,  
SUM(total_amount) total_spent  
FROM orders  
GROUP BY customer_id  
HAVING SUM(total_amount) > 50000;
```

Problem 15

Cities Having More Than One Customer

```
SELECT
city,
COUNT(*) customer_count
FROM customers
GROUP BY city
HAVING COUNT(*) > 1;
```

Problem 16

Products Sold More Than Once

```
SELECT
product_id,
SUM(quantity)
FROM order_items
GROUP BY product_id
HAVING SUM(quantity) > 1;
```

Problem 17

Customer Names With Orders

```
SELECT
c.customer_name,
o.order_id
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id;
```

Problem 18

Product Names For Each Order

```
SELECT
o.order_id,
p.product_name
FROM orders o
```

```
JOIN order_items oi
ON o.order_id = oi.order_id
JOIN products p
ON oi.product_id = p.product_id;
```

Problem 19

Customer Name And Order Amount

```
SELECT
c.customer_name,
o.total_amount
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id;
```

Problem 20

Total Spending By Customer

```
SELECT
c.customer_name,
SUM(o.total_amount) total_spending
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

Problem 21

Total Sales By Category

```
SELECT
p.category,
SUM(p.price * oi.quantity) total_sales
FROM products p
JOIN order_items oi
ON p.product_id = oi.product_id
GROUP BY p.category;
```

Problem 22

Order Count Per Customer

```
SELECT
c.customer_name,
COUNT(*) total_orders
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

Problem 23

Customers Spending More Than ₹40,000

```
SELECT
c.customer_name,
SUM(o.total_amount) total_spending
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name
HAVING SUM(o.total_amount) > 40000;
```

Problem 24

Categories Generating More Than ₹50,000

```
SELECT
p.category,
SUM(p.price * oi.quantity) revenue
FROM products p
JOIN order_items oi
ON p.product_id = oi.product_id
GROUP BY p.category
HAVING SUM(p.price * oi.quantity) > 50000;
```

Problem 25

Customers Placing More Than One Order

```
SELECT
c.customer_name,
COUNT(*) order_count
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name
HAVING COUNT(*) > 1;
```

Topics Covered in Problems 1–25

```
SELECT
WHERE
ORDER BY
SUM
AVG
MIN
MAX
COUNT
GROUP BY
HAVING
INNER JOIN
Aggregate Functions
Business Reporting
Customer Analytics
Product Analytics
Sales Analytics
```

The next panel (Problems 26–50) covers:

```
Subqueries
Correlated Subqueries
Derived Tables
CTEs
CASE
COALESCE
String Functions
Date Functions
Advanced Joins
Nested Aggregations
```

which is where most SQL interview questions start becoming medium-to-hard difficulty.